

Homework 4

Total Marks: 70

Deadline: April 28th 2026

General Instructions: You may partner with up to three other students (**a group of max 4**) to submit a single write-up in a single group. We encourage discussion with other students, even if they are not in your group. Each student must build a good understanding of all the questions even if they discuss and collaborate with others. Merely dividing the problems among group members and putting those together before submission will severely impact your learning, and we **strongly advise against it**. Furthermore, while it is okay to use online resources, including **Generative AI** such as **ChatGPT**, for learning purposes, **blatant copying** or **mindlessly querying** the Generative AI is **forbidden**. If there is any confusion or questions, post those on the **WhatsApp** Group or see your TA or Instructor. **Submission Instructions:** Typed or hand-written solutions (only in PDF format) can be submitted on LMS before the deadline. Email submissions WILL NOT be accepted. There will be NO late days or extensions. *Only one submission per group is needed; the first page must clearly mark the names and students IDs of all group members.*

Question 1 SegWit Fundamentals: Transaction Structure, Malleability, and Witness Programs (10)

Segregated Witness (SegWit) changed the way digital signatures and script arguments are stored inside Bitcoin transactions.

- a)** Explain what data SegWit removes from the traditional scriptSig and where this data goes in a SegWit transaction. Why does this separation reduce transaction malleability? (2)
- b)** Consider a SegWit pay-to-witness-public-key-hash (P2WPKH) output with witness program:

0x00 || HASH160(pubkey).

Construct the spending transaction fields:

- The scriptSig of the input,
- The witness stack items,
- A short explanation of how the P2WPKH script is evaluated. (4)

- c)** SegWit introduces the concept of “witness weight” and defines the block weight metric:

$$\text{weight} = (\text{non-witness bytes} \times 4) + (\text{witness bytes} \times 1).$$

Explain why witness data is discounted and give one practical effect on transaction fee incentives. (2)

d) Describe how SegWit enables future extensibility of script versions without requiring a hard fork. Provide one example of how witness versioning allows new script types to be introduced in a backward-compatible way. (2)

Question 2 Bitcoin Scaling for Payments and Privacy Considerations (12)

Bitcoin's base layer has limited throughput but supports several techniques to scale payments and improve user privacy.

a) Describe why Bitcoin's base layer cannot scale to global, high-frequency payments. Include discussion of block size, block interval, and validation constraints. (2)

b) Explain how the Lightning Network improves payment scalability. In your answer, describe:

- how off-chain payments reduce on-chain load,
- how multi-hop routed payments work, and
- the role of HTLCs in atomic forwarding. (3)

c) Identify two privacy weaknesses of base-layer Bitcoin transactions related to:

- a. input/output linkability,
- b. address reuse. (2)

d) Give two techniques—wallet-level or protocol-level—that improve privacy for everyday Bitcoin payments. Explain briefly how each helps. (2)

e) Compare the privacy characteristics of on-chain payments versus off-chain Lightning payments. In particular, who learns the payment details in each case, and what information is hidden from observers? (3)

Question 3 MAST (Merkelized Abstract Syntax Trees) and Script Commitments (10)

Taproot uses MAST to commit to multiple possible spending scripts while revealing only the branch that is actually executed. This greatly improves privacy and reduces on-chain data.

a) Explain the key idea behind MAST. In your answer, describe how Taproot commits to a set of possible scripts, how only the executed script branch is revealed, and why this provides better privacy than traditional P2WSH script commitments. (3)

b) Consider a Taproot output with three possible spending conditions represented by the following witness scripts:

$$S_1 = \langle \text{pkA} \rangle \text{ OP_CHECKSIG},$$

$$S_2 = \text{OP_IF } \langle \text{pkB} \rangle \text{ OP_CHECKSIG OP_ELSE } \langle \text{pkC} \rangle \text{ OP_CHECKSIG OP_ENDIF},$$

$$S_3 = \langle \text{pkD} \rangle \ 2 \ \text{OP_CHECKMULTISIG}.$$

Construct the MAST (Merkle tree) over the *script hashes* of these three scripts. (2)

- Compute the leaf hashes $h_1 = H(S_1)$, $h_2 = H(S_2)$, $h_3 = H(S_3)$.
- Draw the Merkle tree assuming a standard taptree-style Merkle construction (no leaf duplication).
- Compute the internal node hashes and the final Merkle root M .

c) Suppose the spender uses only script S_2 . Describe exactly what information must be included in the witness for a script-path Taproot spend, and show the Merkle proof path the spender must provide using the tree you constructed in part (b). (3)

d) Explain why, even though three scripts exist, a chain observer learns nothing about S_1 or S_3 when S_2 is used for the spend. (2)

Question 4 HTLC Scripts and Time-Locked Spending Under Taproot (12)

Hashed Time-Locked Contracts (HTLCs) allow conditional payments based on the revelation of a preimage or on a timeout refund. Taproot enables implementing HTLCs either through script-path spending (with a taptree leaf) or through cooperative key-path spending when both parties agree.

a) Explain the purpose of an HTLC in a multi-hop payment setting (e.g., Lightning). In your answer, describe the roles of the hashlock and timelock, and why the timelock must be *relative* or *absolute* depending on the protocol design. (2)

b) Consider a Taproot output between Alice and Bob with internal key P and a taptree consisting of a single script leaf representing the HTLC. Let the payment hash be $h = H(x)$ and let the timeout be enforced by a timelock T . Construct a Taproot-compatible script-path HTLC leaf script that allows:

- Alice to spend the output *if she learns the preimage x before time T* , and
- Bob to refund the output *after time T* .

You may assume standard opcodes such as:

OP_CHECKSIG, OP_HASH160, OP_EQUAL, OP_CHECKLOCKTIMEVERIFY, and OP_DROP. (3)

c) Draw the Taproot taptree for this output (one script leaf), label the leaf hash, and show how it combines with the internal key P to form the tweaked Taproot output key Q . (2)

d) Suppose the HTLC output is spent cooperatively via the *key-path* with a single Schnorr signature. What information does a blockchain observer learn about:

- a. whether this was an HTLC at all,
- b. whether a timelock existed,
- c. whether a hash preimage was involved?

Explain why Taproot provides this level of privacy. (2)

e) Suppose Alice spends via the script-path before time T using the preimage x . Write the full witness stack (in logical order) required for this Taproot script-path spend, including the Merkle proof/control block. (3)

Question 5 Lightning Revocation Mechanisms and Punishment for Old-State Broadcasts (12)

Two-party payment channels require both parties to update their channel balances off-chain while preventing either participant from broadcasting an outdated commitment transaction. The Lightning Network uses a revocation-based punishment mechanism to enforce correct behavior.

a) Explain why outdated channel states (old commitment transactions) are dangerous. Describe what would happen if a party could safely publish an earlier state without penalty. (2)

b) In the simplified Lightning model used in class, each commitment transaction contains a *revocation key* whose private secret is revealed only when that state becomes old. Explain how a new state revokes the prior one. In your explanation, include:

- the role of the revocation secret for each state,
- how the counterparty uses it to gain control of the revocation key,
- and why revealing this secret makes an older state unsafe to publish. (3)

c) Suppose Alice and Bob open a channel. When they sign state $i + 1$, Alice reveals the revocation secret for state i , and Bob stores it. If Alice later broadcasts state i , which output can Bob take, and why? Describe how the timelock on that output enables Bob to punish Alice. (3)

d) Construct a simplified script (in Bitcoin Script) for a *revocable output* that pays either:

- the original owner after a timelock expires, or
- the counterparty immediately if they know the revocation secret. (2)

e) Describe what Bob includes in his *punishment (justice) transaction* when Alice broadcasts an old state, and explain why this mechanism creates strong economic incentives to only broadcast the latest state. (2)

Question 6 Lightning Network Fundamentals: Payment Channels and Commitment Transactions (14)

Two parties, Alice and Bob, set up a bidirectional payment channel using a funding transaction and off-chain commitment transactions representing their evolving balances.

a) Describe the structure and purpose of the *funding transaction* in a Lightning channel. Why must it be a 2-of-2 multisig output, and how does this fact enable trust-minimized updates? (3)

b) Define what a *commitment transaction* is. Explain:

- who signs it,
- when it is valid to broadcast,
- why each party holds a slightly different commitment transaction. (3)

c) Explain how off-chain payments change the channel balance without broadcasting transactions. Describe the update process that ensures both parties end up with the latest valid state and that older states cannot be safely broadcast. (3)

d) Consider a channel where Alice initially contributes 1 BTC and Bob contributes 0 BTC. They then perform the following off-chain transfers:

- Alice pays Bob 0.3 BTC,
- Bob pays Alice 0.1 BTC,
- Alice pays Bob 0.25 BTC.

Compute the final off-chain channel balance for each party and explain how this is reflected in the final pair of commitment transactions. (3)

e) Give one reason why commitment transactions must include timelocks, and describe what could go wrong if timelocks were not present. (2)